

LAB – Week 2 (Numpy and Pandas Lab Manual)

Please take the time to watch this video for a 120-minute session to enhance your understanding of Numpy and pandas.

<https://www.youtube.com/watch?v=GB9ByFAIAH4>

<https://www.youtube.com/watch?v=vmEHCJofslg&t=2182s>

Welcome to the Numpy and pandas Lab! In this lab, you will learn how to work with Numpy arrays and pandas Dataframes. Follow the instructions in each exercise and complete the coding tasks. Feel free to ask your instructor if you have any questions.

Exercise 1: Introduction to Numpy

1. Create a new Python file named `numpy\_intro.py`.
2. Inside the file, import the Numpy library.
3. Create a 1D Numpy array with the elements: 10, 20, 30, 40, 50.
4. Create a 2D Numpy array with the following elements:

```
[[1, 2, 3],  
 [4, 5, 6],  
 [7, 8, 9]]
```
5. Create a 3D Numpy array with the following elements:

```
[[[1, 2],  
   [3, 4]],  
 [[5, 6],  
   [7, 8]]]
```
6. Use the `.ndim` method to get the dimension of each array and print the results.
7. Save the file and run the program to check if the dimensions of each array are displayed correctly.

Exercise 2: Array Indexing and Slicing

1. Create a new Python file named `numpy\_indexing.py`.
2. Inside the file, define a 1D Numpy array with the elements: 10, 20, 30, 40, 50.

3. Use indexing to access the second element (20) of the array and print the result.
4. Use slicing to access a subset of the array containing elements from the second to fourth index.
5. Save the file and run the program to check if the correct elements are accessed.

Exercise 3: Array Shape and Reshaping

1. Create a new Python file named ``numpy_shape.py``.
2. Inside the file, define a 2D Numpy array with the following elements:

```
[[1, 2, 3],  
 [4, 5, 6],  
 [7, 8, 9]]
```
3. Use the ``shape`` method to get the shape of the array and print the result.
4. Use the ``reshape()`` method to reshape the 2D array into a 1D array and print the result.
5. Save the file and run the program to check if the shape and reshaped array are displayed correctly.

Exercise 4: Array Sorting and Finding

1. Create a new Python file named ``numpy_sort.py``.
2. Inside the file, define a 1D Numpy array with the elements: 30, 10, 50, 20, 40.
3. Use the ``sort()`` method to sort the array in ascending order and print the result.
4. Use the ``max()`` and ``min()`` methods to find the maximum and minimum values in the array and print the results.
5. Use the ``where()`` function to find the indices where the elements in the array are greater than 30 and print the results.
6. Save the file and run the program to check if the correct sorting and finding operations are performed.

Exercise 5: Numpy Statistical Functions

1. Create a new Python file named ``numpy_statistics.py``.

2. Inside the file, define a 2D Numpy array with the following elements:

```
[[1, 2, 3],
```

```
[4, 5, 6],
```

```
[7, 8, 9]]
```

3. Use the `np.mean()` function to find the mean of the elements in the array and print the result.

4. Use the `np.std()` function to find the standard deviation of the elements in the array and print the result.

5. Save the file and run the program to check if the mean and standard deviation are displayed correctly.

Exercise 6: Numpy Array Flattening and Stacking

1. Create a new Python file named `numpy_flatten_stack.py`.

2. Inside the file, define a 2D Numpy array with the following elements:

```
[[1, 2, 3],
```

```
[4, 5, 6]]
```

3. Use the `.flatten()` method to flatten the 2D array into a 1D array and print the result.

4. Create another 2D Numpy array with the following elements:

```
[[7, 8, 9],
```

```
[10, 11, 12]]
```

5. Use the `np.vstack()` function to vertically stack the two 2D arrays and print the result.

6. Use the `np.hstack()` function to horizontally stack the two 2D arrays and print the result.

7. Save the file and run the program to check if the flattening and stacking operations are performed correctly.

Exercise 7: Introduction to Pandas

1. Create a new Python file named `pandas_intro.py`.
2. Inside the file, import the Pandas library.
3. Create a DataFrame using a dictionary with the following data:

```
{  
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],  
  'Age': [25, 30, 22, 28, 35],  
  'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']  
}
```
4. Use the `.head()` method to display the first few rows of the DataFrame and print the result.
5. Save the file and run the program to check if the DataFrame is displayed correctly.

Exercise 8: Accessing DataFrame Columns and Rows

1. Create a new Python file named `pandas_access.py`.
2. Inside the file, define a DataFrame with the following data:

```
{  
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],  
  'Age': [25, 30, 22, 28, 35],  
  'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']  
}
```
3. Use indexing to access the 'Name' column of the DataFrame and print the result.
4. Use the `.loc[]` method to access the first row of the DataFrame and print the result.
5. Save the file and run the program to check if the column and row are accessed correctly.

Exercise 9: Accessing Multiple Columns and Rows in DataFrame

1. Create a new Python file named `pandas_multiple.py`.
2. Inside the file, define a DataFrame with the following data:

```
{  
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],  
  'Age': [25, 30, 22, 28, 35],  
  'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']  
}
```
3. Use indexing to access multiple columns of the DataFrame and print the result.
4. Use the `.loc[]` method to access multiple rows of the DataFrame and print the result.
5. Save the file and run the program to check if the multiple columns and rows are accessed correctly.

Exercise 10: DataFrame Subset Selection

1. Create a new Python file named `pandas_subset.py`.
2. Inside the file, define a DataFrame with the following data:

```
{  
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],  
  'Age': [25, 30, 22, 28, 35],  
  'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']  
}
```
3. Use the `.iloc[]` method to access a specific row and column of the DataFrame and print the result.
4. Use slicing to select a subset of the DataFrame and print the result.
5. Save the file and run the program to check if the subset selection is performed correctly.